

AnomalyIQ

Statistical anomaly detection for tabular data — technical report

Dishank Shah · B.Tech Computer and Communication Engineering
Manipal Institute of Technology

Live: <https://anomalyiq.vercel.app> · Source: <https://github.com/Dishank13/anomalyiq>

How to read this document. Sections 1–3 describe what the system is and how it is put together. Sections 4–6 are the engineering substance — the detection mathematics, the measured benchmark, and the job pipeline — and are where the interesting interview questions live. Section 9 collects the bugs that were found and fixed, which are often the most useful thing to be able to talk about: they show how you find problems, not just how you write code.

1. Project Overview

AnomalyIQ is a production-deployed web platform that finds statistical anomalies in tabular data and explains them in plain English. A user uploads a CSV or Excel file, chooses which columns to monitor and which detection methods to apply, and the system runs the analysis as a background job, streaming progress to the browser and marking each finding on a chart of the underlying series.

Each finding carries the value, the range it was expected to fall in, its z-score, the method that caught it, a severity rating, and a one-sentence explanation generated by Google Gemini.

The system is built as four services — a React frontend, a Node.js API, a Python detection service, and a worker — communicating over HTTP, WebSockets and a Redis job queue, with MongoDB for persistence. Each is containerised and deployed independently.

What makes it more than a CRUD app

- **The detection methods are measured, not asserted.** Four detectors are scored against synthetic data with known anomaly positions, producing a precision/recall/F1 table that justifies which method is used when.
- **Analysis is an asynchronous job** with idempotency, retries with exponential backoff, progress events and a persisted audit trail — not a blocking HTTP request.
- **Correctness was pursued explicitly.** A statistical masking bug, a cross-tenant data leak, and a destructive non-atomic write were each found, quantified and fixed.

2. Problem Statement

Most people working with data monitor it by eye and hope they notice when something goes wrong. The existing tools sit at two extremes.

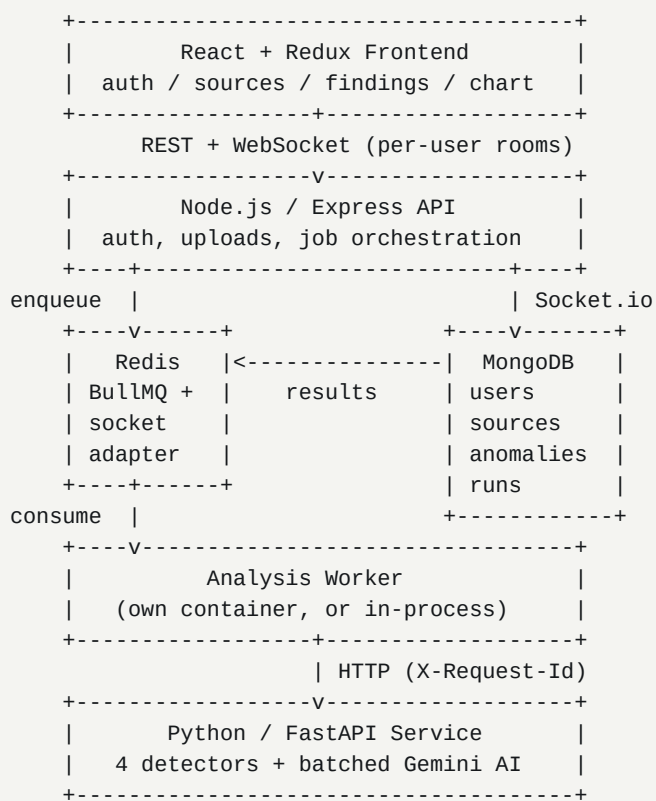
Approach	Limitation
Spreadsheets (Excel, Sheets)	You eyeball a chart and catch anomalies by chance. There is no notion of what "expected" means for a given point.

Approach	Limitation
Observability platforms (Datadog, Grafana, New Relic)	Built for server metrics and DevOps teams. Significant configuration, real cost, and they do not ingest an arbitrary spreadsheet.

AnomalyIQ sits in the gap: free and self-hostable, works on any tabular data rather than just time-series server metrics, uses interpretable statistical methods rather than an opaque model, and explains each finding in a sentence instead of drawing a red line and leaving you to work it out.

The honest limitation. The detectors are unsupervised and have no knowledge of the domain. They identify values that are statistically unusual, which is not the same as values that are *wrong*. A legitimate Black Friday spike is an anomaly by every measure here. The tool narrows where to look; it does not make the judgement.

3. System Architecture



Services

Service	Technology	Responsibility
Frontend	React, Redux, Tailwind, Recharts	UI, state, live updates, charting
API	Node.js, Express, Socket.io	Auth, uploads, job orchestration, WebSockets
Worker	Node.js, BullMQ	Consumes analysis jobs, retries, emits progress
Detection	Python, FastAPI, pandas, numpy	Statistics, Gemini calls
Database	MongoDB + Mongoose	Users, sources, anomalies, run history
Queue / bus	Redis	BullMQ backbone, Socket.io adapter

The request path, end to end

- The browser uploads a file. The API validates type and size, stores the bytes base64-encoded in MongoDB, and asks the Python service to report its columns.
- The user triggers analysis. The API validates the source, derives a deterministic job id, writes an `AnalysisRun` record, enqueues the job and returns `202 Accepted` with the job id.
- The worker picks up the job and calls the Python service, forwarding the originating request id so one analysis is traceable across all three services in the logs.
- Python loads the file, runs the selected detectors, and asks Gemini to explain the top findings in a single batched call.
- The worker saves the findings and emits progress and completion events into the requesting user's Socket.io room. The browser updates without a refresh.

Why microservices, honestly

A monolith would work for this workload. Two reasons made the split worth it, and one commonly-cited reason did not apply.

- **Language fit.** pandas, numpy, statsmodels and scikit-learn have no equivalent in Node. Writing STL decomposition or Isolation Forest in JavaScript would mean reimplementing mature libraries badly.
- **Failure isolation.** The detection service is the part that does heavy, slow, memory-hungry work. When it is cold or overloaded, the API stays responsive and reports the job as retrying rather than falling over.
- **Independent scaling was *not* a real driver** at this traffic level. Claiming it would be would be dishonest; the split earns its keep on the two points above.

4. Anomaly Detection — Mathematics

Four detectors are available and selectable per analysis. Each covers a failure mode the others are blind to.

4.1 Z-score over a causal rolling window

The z-score measures how many standard deviations a value sits from the mean. For a value x with mean μ and standard deviation σ :

$$Z = (x - \mu) / \sigma$$

A value beyond three standard deviations is flagged. Under a normal distribution only about 0.3% of data falls beyond ± 3 sigma.

Why a rolling window

A global mean assumes "normal" never changes. It does. A stock at \$100 has different normal behaviour than the same stock at \$180, and a global mean would either miss later anomalies or flag ordinary growth. A rolling window of 30 points computes what is normal for the recent past, making detection adaptive to trend.

Why the window must be causal — and the bug that was here

The original implementation computed the rolling statistics over a window that *included the point being tested*. This is a real statistical error called masking: an extreme value inflates the standard deviation it is then measured against, and so partially conceals itself. The more extreme the outlier, the more it hides.

```
# before -- the point under test is inside its own reference window
rolling_mean = series.rolling(window=30, min_periods=5).mean()

# after -- shift by one so only prior points inform the estimate
rolling_mean = series.shift(1).rolling(window=30, min_periods=5).mean()
```

The fix raised mean F1 across anomaly regimes from 0.35 to 0.39. It also makes the detector strictly more sensitive, which costs false positives on trending data — reported rather than hidden, and explained in section 5.

4.2 IQR (interquartile range)

IQR splits the data into quartiles and flags values outside the fences:

```
IQR          = Q3 - Q1
lower fence = Q1 - 1.5 x IQR
upper fence = Q3 + 1.5 x IQR
```

IQR is non-parametric — it assumes nothing about the distribution — and is resistant to extreme values, because quartiles do not move when one point goes to infinity. That makes it complementary to the z-score, whose sigma is dragged by outliers.

4.3 STL residual (seasonal decomposition)

Seasonal-Trend decomposition using LOESS splits a series into trend, seasonal and residual components. Only the residual — the part the seasonal model cannot explain — is tested.

This catches the case both methods above are blind to: a value that is perfectly ordinary in magnitude but wrong *for its position in the cycle*. Weekday traffic appearing on a Sunday is within the normal range and inside the quartile fences; it is only anomalous relative to the cycle.

A calibration detail worth knowing. The residuals are scaled by the median absolute deviation (MAD), which measures the tight core of a robustly fitted residual distribution and runs far narrower than a standard deviation. Those z-scores are therefore *not* on the familiar 3-sigma scale — at a threshold of 3 the detector flagged 39 ordinary noise points per clean series. The threshold was tuned on the benchmark to 8, which gives F1 0.97 on seasonal anomalies with zero false positives on clean data.

4.4 Isolation Forest (multivariate)

Every method above is univariate: each column is judged alone. None can see a row whose individual values are all unremarkable but whose *combination* is impossible — an order with a normal quantity and a normal unit price that together imply a total that cannot occur.

Isolation Forest scores whole rows by how few random splits it takes to isolate them. Each flagged row is attributed to whichever of its columns deviates most, so the result still names a column and reads like every other finding.

4.5 Severity

Z	Severity	Statistical rarity under normality
> 5.0	High	Extremely rare — under 0.00003% of values
3.5 – 5.0	Medium	Very rare — under 0.05%
3.0 – 3.5	Low	Rare — under 0.3%

4.6 Why not deep learning

LSTM autoencoders, ARIMA and similar methods exist and are stronger on some problems. They were rejected deliberately: they require training data the user does not have, hyperparameter tuning per dataset, and significant compute — and crucially they are not interpretable. This product must be able to say *why* a value was flagged, in a sentence. "The model assigned it a high reconstruction error" is not that. For a general-purpose detector that must work on an arbitrary uploaded spreadsheet with no training phase, statistical methods are the right tool.

5. The Benchmark

"We use z-score and IQR" is an implementation note. The question a reviewer should ask is: how do you know they work, and how do you know which to use? The benchmark answers that.

5.1 Method

Synthetic series are generated whose anomaly positions are known by construction, across five regimes each designed to defeat a different detector. Every detector is scored on every regime at three noise levels. A detection counts as a true positive if it lands within two positions of a labelled anomaly, and each labelled anomaly can be matched only once — so a detector cannot inflate recall by firing repeatedly around a single event.

5.2 Results — F1 by regime and method

Regime	z-score	IQR	STL	Isolation Forest
point spikes	0.76	0.67	0.94	n/a
level shift	0.50	0.00	0.00	n/a
seasonal break	0.00	0.00	0.97	n/a
variance change	0.29	0.10	0.20	n/a
multivariate	0.11	0.20	0.00	0.70

No column wins everywhere, which is the entire argument for shipping several detectors and letting the user choose. Two results matter most:

- STL scores **0.97** on seasonal anomalies that z-score and IQR both score **0.00** on. Those anomalies are invisible to magnitude-based tests by construction.
- Isolation Forest scores **0.70** on multivariate anomalies where the best univariate method manages **0.20** — a 3.5x improvement on a failure mode no single-column method can address.

5.3 False positives on clean data

F1 is meaningless on a control series with no anomalies: what matters is how many findings a detector invents. Reported separately for that reason.

Method	Mean false positives per 500-point series
z-score	6.3
IQR	0.0
STL	0.0

The z-score number is not random noise — it is the trend. A causal rolling mean necessarily *lags* a rising series, and that systematic offset reads as deviation. Removing the trend from the control series drops it from about 6 to about 2, which is the level the t-distribution predicts for an estimated sigma over a 30-point window. STL scores zero precisely because it removes trend and seasonality before testing anything — another reason the two are complementary rather than redundant.

5.4 Does combining detectors help?

Configuration	Mean F1
z-score only	0.39
z-score + IQR (the original shipped default)	0.31
all three univariate methods	0.55

Adding detectors raises recall and lowers precision, and which wins depends on the data. Note the middle row: **the configuration that originally shipped scored worse than z-score alone**. That is exactly the kind of thing that is invisible without measurement, and it is why methods are now selectable per analysis rather than fixed.

5.5 Performance

Detection originally iterated every row in Python. Vectorising it with numpy masking produced identical output, verified row for row:

Rows	Original	Vectorised	Speed-up
10,000	0.124 s	0.002 s	71x
100,000	1.354 s	0.022 s	61x
1,000,000	14.017 s	0.194 s	72x

The benchmark doubles as a regression test. It runs under pytest with a minimum F1 asserted per detector, so a change that degrades the mathematics fails rather than silently producing worse results. Reproduce with `pytest bench/` and `python -m bench.evaluate`.

6. The Analysis Pipeline

Analysis originally ran inside the HTTP request and could hold it for up to 90 seconds. On a cold free-tier instance that is indistinguishable from a frozen browser. It is now a background job.

6.1 The flow

```
POST /api/anomalies/analyze/:sourceId
-> validate source, ownership, file availability (fast, in-request)
-> jobId = sourceId + sha256(file) + hash(options)
-> persist AnalysisRun { status: queued }
-> enqueue on BullMQ
<- 202 Accepted { jobId, status: "queued" }

worker
-> analysis:running          -> socket, user room only
-> call python service       -> X-Request-Id forwarded
-> analysis:progress 15/75   -> socket
-> insert findings, prune previous run
-> analysis:completed        -> socket

GET /api/anomalies/runs/:jobId -- poll fallback if the socket drops
```

6.2 The properties that matter

Property	How, and why it is needed
Idempotent	The job id is derived from the source, a SHA-256 of the file contents, and a hash of the analysis options. Double-clicking "Run analysis" joins the existing job rather than racing it; changing any option deliberately produces a different job.
Retried	Three attempts with exponential backoff. A cold Python service that fails the first call answers fine on the second. Failures are retained for 24 hours, successes for one.
Observable	Every run is persisted as an AnalysisRun with status, progress, stage, timings and error. A dropped websocket or a closed tab no longer makes a run unobservable.
Degradable	With no REDIS_URL the same runner executes inline in the request and emits the same events. The app is fully functional without Redis, so a lapsed free-tier add-on degrades latency instead of causing an outage.

6.3 One worker, two topologies

The worker is a separate module with its own endpoint. Under docker-compose it runs as its own container — the real topology. On free-tier hosting it runs inside the API process behind a flag, because a second always-on service is a second thing to keep awake. The same code runs in both; the Socket.io Redis adapter is what lets a separate worker process reach browser connections held by the API.

Interview note. The single most important design choice here is that both the queued path and the inline fallback call *one* shared runner. Whether Redis is configured changes *when* the work happens, never *what* happens — which is what stops the fallback path from rotting unnoticed while nobody exercises it.

7. Technology Choices and Alternatives

Choice	Why	Alternative considered
React + Redux Toolkit	Component model suits repeated UI pieces (findings, badges, charts); one store avoids prop-drilling auth and data through the tree.	Context API — simpler, but causes unnecessary re-renders as state grows.
Node.js + Express	Event-driven I/O suits an orchestration layer juggling uploads, sockets and upstream calls concurrently. First-class Socket.io.	Django — one language overall, but weaker real-time story.
Python + FastAPI	The data-science ecosystem has no equivalent elsewhere. Async by default, Pydantic validation, generated OpenAPI docs.	Flask — synchronous, needs extra work for the same validation and concurrency.
MongoDB + Mongoose	Data shapes genuinely vary — a CSV source and a stock source hold different config. Mongoose adds schema validation and middleware hooks.	PostgreSQL — better for rigid relational data; the flexible config objects fit documents more naturally.
Socket.io	Server pushes findings the moment they exist. Handles reconnection and falls back to long-polling. Per-user rooms make isolation explicit.	Server-Sent Events — simpler but unidirectional, no built-in rooms.
BullMQ + Redis	Mature Node job queue with retries, backoff, and job-level idempotency via job ids.	In-process setTimeout — no durability, no retries, dies with the process.
JWT + bcrypt	Stateless auth scales horizontally with no shared session store. Passwords are stored only as bcrypt hashes at 12 salt rounds.	Server sessions — need shared storage between instances.
Google Gemini	Generous free tier, no card required, output quality fine for three sentences of analysis prose.	OpenAI — comparable quality, but paid from the first request.

8. Key Features and Interface

8.1 Authentication

Email and password, hashed with bcrypt at 12 salt rounds — passwords are never stored. JWTs expire after seven days. An axios interceptor attaches the token to every request, and a route guard redirects unauthenticated users. Login and registration are rate limited, and both return the same message for an unknown email and a wrong password, so the endpoint cannot be used to discover who has an account.

8.2 Upload and analysis

CSV, XLSX and XLS up to 8 MB. Files are held in memory and persisted base64-encoded in MongoDB rather than on disk, because the container filesystem is wiped on every restart and redeploy — which previously made analysis fail for every source uploaded before the last restart. Text columns that are really numbers (thousands separators, currency symbols) are coerced so they are analysed rather than skipped.

8.3 The chart

The series is plotted with every finding marked. Selecting a finding reveals the exact range that point violated. There is deliberately no continuous expected-range band: the rolling bounds are only known at the points the analysis actually tested, so drawing one everywhere would mean interpolating limits that were never computed.

8.4 Interface design rules

- **Colour encodes severity and nothing else.** Buttons, links, focus rings and chrome are all ink. In a tool whose job is to pull your eye to outliers, a coloured button competes with the signal.
- **The severity palette was validated, not eyeballed** — worst adjacent pair deltaE 18.6 under deuteranopia, 22.1 under normal vision, against the page surface.
- **Severity never rides on colour alone.** Every finding carries a crosshair glyph and a text label as well, so the interface works for colour-blind users and in print.
- **Every numeral is monospaced with tabular figures**, so a column of z-scores can be compared by scanning it.

9. Defects Found and Fixed

These are worth studying: each was found a different way, and being able to describe how you found a problem is usually more persuasive than describing a feature.

9.1 Silent feature death — the retired AI model

Symptom: none. The UI showed plausible text under an "AI Explanation" heading.

Cause: Google retired `gemini-2.0-flash`. Every call returned 404, and the exception handler fell back to a deterministic template string. Nothing in the logs distinguished this from normal operation, so the flagship feature was dead in production and nobody knew.

Fix: a "-latest" model alias so a retirement cannot silently break it; a health endpoint that reports whether real AI text or fallback text is being served; and configuration errors logged loudly and distinctly from transient ones.

Lesson: a fallback that is indistinguishable from success is a bug, not resilience.

9.2 Wrong architecture, not a wrong value — the quota

Fixing the model exposed the real problem: the code made one API call per anomaly, against a free tier of five requests per minute. A 25-anomaly analysis exhausted the quota in seconds. Batching all findings into a single request with a typed JSON response schema took it from 25 requests to **1**, and from 2-of-56 explained to 25-of-25 in about 13 seconds.

9.3 Cross-tenant data leak

The server broadcast findings with `io.emit()`, which sends to every connected socket. The client sent an auth token in its handshake; the server never checked it. Every logged-in browser received every other user's findings — the frontend merely hid the ones it did not recognise, so the data still crossed the wire. Fixed by verifying the handshake and joining each socket to a per-user room. Verified with two concurrent sessions.

9.4 A uniqueness guarantee that was not there

Registration guarded duplicate emails with a find-then-create check, which loses the race between two concurrent signups. Removing it exposed that the unique index was the only real constraint — and Mongoose builds indexes automatically only when `autoIndex` is on, which production deployments routinely disable. Indexes are now built explicitly at startup. Four concurrent signups for one email now create exactly one account.

9.5 Destructive non-atomic write

Saving results deleted the previous run *before* inserting the new one. When a stale enum rejected a newly-added detection method, the delete had already succeeded and the insert failed — destroying all 95 existing findings and leaving nothing. Reordered to insert first and prune after, which is recoverable in both directions: a failed insert leaves the previous results intact, a failed prune leaves duplicates the next run clears.

How it was found: by running the application and clicking the feature, not by reading the code. The unit path was tested; the integration path was not.

9.6 Statistical masking

Covered in section 4.1: the rolling window included the point under test, inflating the sigma that point was measured against. Found by building a benchmark that could measure the difference, and quantified at +0.04 mean F1.

10. Deployment and Operations

Component	Platform	Notes
Frontend	Vercel	Automatic deploy on push to main
API	Render	Docker; free tier sleeps after 15 min idle
Detection service	Render	Docker; cold start ~50 s
Database	MongoDB Atlas	AWS Mumbai
Queue	Optional	Inline fallback when REDIS_URL is unset

10.1 Observability

- **Structured logging** (pino) with tokens and file contents redacted.
- **Request correlation.** Each request gets an id, forwarded to the Python service as X-Request-Id, so one analysis can be followed across all three services.
- **Liveness and readiness** are separate: /healthz asks only whether the process is up, so a degraded dependency cannot get the container restarted; /readyz reports MongoDB, the detection service, Redis, and which analysis mode is active.

10.2 Security posture

- bcrypt at 12 salt rounds; passwords never stored in any recoverable form.
- helmet security headers; rate limits on the API broadly and credentials specifically.
- Request bodies validated with zod before reaching any handler.
- Socket connections authenticated at handshake and confined to per-user rooms.
- Errors return generic messages; driver and schema internals are logged, never sent.
- Secrets supplied by environment only; the repository ships an .env.example.

10.3 Known limitations

- **8 MB upload ceiling.** Files are base64-encoded into MongoDB documents, which are capped at 16 MB, and base64 inflates by about a third. Object storage or GridFS would lift this.
- **Free-tier cold starts.** The first request after idle takes about 50 seconds. The interface says so rather than appearing to hang.
- **No automated CI.** The detection benchmark runs under pytest, but nothing runs it on push yet.
- **The queued path is not exercised in production.** It is code-complete and its application logic is tested, but without a Redis instance the deployment runs the inline fallback.

11. Questions You Should Be Ready For

Study these. Each has a real answer in the sections above.

- *Why z-score and IQR together — are they not redundant?* They fail differently. Z-score is sensitive to the outliers it is trying to find, because sigma is dragged by them; IQR is resistant but has no notion of local trend. Section 5.2 shows the regimes where each wins.
- *What is wrong with a rolling window that includes the current point?* Masking. Section 4.1.

- *Your z-score has 6.3 false positives per clean series. Why?* The trend, not the noise — a causal mean lags a rising series. Section 5.3.
- *Why is the job id a hash of the file rather than a UUID?* Idempotency. A UUID would make every click a new job; the content hash makes an identical re-run collapse onto the job already in flight. Section 6.2.
- *What happens if Redis goes away?* The same runner executes inline. Section 6.2.
- *Why store files in MongoDB rather than on disk?* The container filesystem is ephemeral. Section 8.2 — and the 16 MB document cap is the resulting limitation.
- *How do you know the AI explanations work?* Because the health endpoint reports whether real or fallback text is being served — a lesson learned the hard way. Section 9.1.
- *Why not an LSTM?* No training data, no interpretability, and the product must explain itself. Section 4.6.

Benchmark figures in this document are reproducible from the repository: `pytest bench/`, `python -m bench.evaluate`, `python -m bench.bench_detect`.